VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



VŨ TẤN MINH
VŨ ĐÌNH KIỆT

DESIGN AND PERFORMANCE EVALUATION OF A MICROSERVICES SYSTEM FOR LARGE-SCALE CONCURRENT TRANSACTION PROCESSING IN A FLASH-SALE SCENARIO

MIDTERM PROGRESS REPORT SOFTWARE ENGINEERING

HO CHI MINH CITY, YEAR 2026

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



VŨ TẤN MINH – 523V0012
VŨ ĐÌNH KIỆT – 523V0011

DESIGN AND PERFORMANCE EVALUATION OF A MICROSERVICES SYSTEM FOR LARGE-SCALE CONCURRENT TRANSACTION PROCESSING IN A FLASH-SALE SCENARIO

MIDTERM PROGRESS REPORT
SOFTWARE ENGINEERING

Advised by
Dr. MAI VĂN MẠNH

HO CHI MINH CITY, YEAR 2026

DESIGN AND PERFORMANCE EVALUATION OF A MICROSERVICES SYSTEM FOR LARGE-SCALE CONCURRENT TRANSACTION PROCESSING IN A FLASH-SALE SCENARIO

ABSTRACT

This report presents the midterm progress of the project after Week 12 of the 15-week plan. The system takes flash-sale orders through an asynchronous pipeline. An Order Service, built on .NET 10 and the ABP Framework, writes each order and its `OrderPlaced` event in one PostgreSQL transaction (Transactional Outbox). RabbitMQ carries the events, an Inventory Service reserves stock atomically with a Redis Lua script, and a Process Worker simulates downstream work.

The work planned for Weeks 1–12 is implemented: the six-state order model, idempotent order intake, Outbox and Inbox, bounded retry and dead-letter queues, stuck-order reconciliation, tracing and metrics, overload control at the gateway, the application and the broker, and the required Terraform deployment on AWS EC2 with JMeter on a separate machine. The automated verifiers and the unit tests pass on the assessed code (Section 7); passing tests show the cases they cover, not universal correctness. In a local overload run (23 October) and an EC2 run on the assessed code (24 October), the system refused excess load early, lost no accepted order, and cleared its backlog in under 7 seconds. Open limitations are listed in Section 10.

A first local pilot offered C1 and C2 the same load. At 50 requests/s both kept up. At 200 requests/s, C2 still accepted every request in under 61 ms (p95), but its single Inventory consumer decided only about 70 orders/s, so stock decisions fell a minute behind; C1 decided about 160/s with response times near 1.1 s. The official comparison, the experiments with extra Inventory consumers and the repeated runs remain for Weeks 13–14.

TABLE OF CONTENTS

1 Introduction **1**
1.1 Problem and research question 1
1.2 Objectives 1
1.3 Scope 1

2 Progress against the plan **2**

3 System design **2**
3.1 Architecture 2
3.2 Order state model 3

4 Work completed in Weeks 1–11 **3**

5 Week 12: overload control and deployment **4**
5.1 Load shedding (C3) 4
5.2 Overload, drain and recovery 4
5.3 AWS deployment 8

6 External review after Week 12 **8**

7 Verification **9**
7.1 Assessed code and environment 10

8 Research-question status **11**
8.1 First local pilot: C1 against C2 11
8.2 Design limits that affect the comparison 13
8.3 Pre-pilot settings 13

9 Team contributions **13**

10 Limitations and threats to validity **13**

11 Risks and mitigation **15**

12 Next steps **15**

1. Introduction

1.1 Problem and research question

A flash sale concentrates a sudden burst of orders on a few products with limited stock. Many concurrent requests competing for one inventory record cause lock contention and longer response times, and a non-atomic check-then-update can sell more units than exist (over-selling) [2]. When the request rate exceeds processing capacity, an unbounded system lets queues and latency grow until it fails.

Research question. How does an event-driven design with atomic Redis Lua reservation (configuration C2) compare with a synchronous PostgreSQL atomic-update baseline (C1) in latency, throughput, overload behaviour and complexity, and what do load shedding (C3) and extra Inventory consumers (C4) change? C0, a naive read-then-write implementation, only demonstrates the race condition.

1.2 Objectives

- **Asynchronous processing:** accept an order, store it with its event in one transaction, and answer with the order ID while stock and downstream work continue through RabbitMQ.
- **Inventory consistency:** stock never negative, successful reservations never above the initial stock, at most one reservation per order, and no duplicate deduction from a duplicate message.
- **Load shedding and resilience:** refuse excess load early at the gateway, the application and the broker; retry transient failures a bounded number of times and dead-letter poison messages.
- **Observability and evaluation:** trace every order by correlation ID, order ID and message ID with its state history and stage timestamps, and measure the effect of each design choice.

1.3 Scope

Backend only; clients are simulated with Apache JMeter. All backend components run on one AWS EC2 instance and JMeter on a separate one. Each order has quantity 1. The Process Worker is deterministic and success-only, so the declared `ProcessingFailed` state has no producer. Authentication, payment, compensation and clustering are out of scope, as the brief states.

2. Progress against the plan

Table 1 compares each week's planned slot in the teacher's 15-week roadmap with the date the week was closed in the project's Notion roadmap, where a week closes only when its tests have run and their logs are saved.

Table 1: *Planned slots (teacher's roadmap) and closing dates (Notion roadmap), 2026.*

Week	Planned slot	Main content	Closed with evidence
1	08/08–14/08	Scope, research questions, state model, protocol	01/10
2	15/08–21/08	Docker Compose environment, ERD, database accounts	01/10
3	22/08–28/08	C0 naive and C1 PostgreSQL baselines, JMeter smoke test	01/10
4	29/08–04/09	ERD, API and event contracts, test plan	01/10
5	05/09–11/09	Order Service API, idempotency, state management	01/10
6	12/09–18/09	Transactional Outbox, publisher confirms	01/10
7	19/09–25/09	Inventory Service, Redis Lua, warm-up	02/10
8	26/09–02/10	Stock results, order status, validator	02/10
9	03/10–09/10	Inbox, idempotent consumer, duplicates	05/10
10	10/10–16/10	Bounded retry, dead-letter queues, reconciliation	11/10
11	17/10–23/10	Process Worker, correlation IDs, metrics	14/10
12	24/10–30/10	Load shedding, Terraform and EC2 deployment	22/10

Weeks 1–8 were closed on 1–2 October, three to seven weeks after their planned slots. Most of that code existed by late September, but a week only closes after a fresh-environment build, executed tests and saved evidence, which were completed then. Weeks 9–12 then closed ahead of plan: Week 12 closed on 22 October, before its slot begins. Milestones M1–M5 are complete; M6–M8 (pilot, official runs and submission) remain.

3. System design

3.1 Architecture

All three services are .NET 10 applications on the ABP Framework [9]. A client posts an order through the Nginx gateway to the Order Service, which writes the order and an `OrderPlaced` Outbox row in one transaction and answers 201 immediately (Figure 1). The Outbox publisher sends the event to RabbitMQ with mandatory routing and publisher confirms [1, 3]. The Inventory Service reserves one unit atomically with a Redis

Lua script [6] and publishes `StockReserved` or `StockRejected` through its own Outbox. The Order Service and the Process Worker consume the result. The Order and Inventory consumers record every message ID in an Inbox table in the same transaction as the business change, so a redelivered message has no second effect. The stateless Worker gives its events fixed message IDs, so Order Service's Inbox drops the repeats.

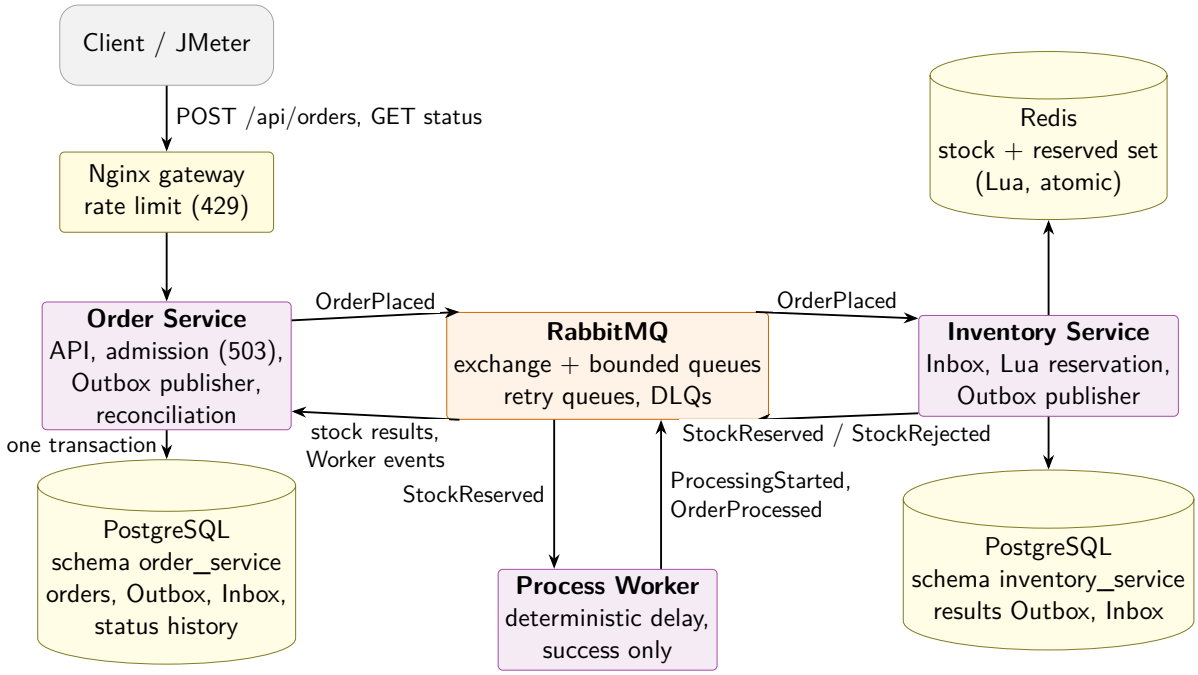


Figure 1: *System architecture and event flow.*

Each service owns a schema in one PostgreSQL instance and connects with its own database account; cross-schema access is denied. This was re-checked on the local stack after the latest migrations (test case TP-E04, 64 of 64 checks).

3.2 Order state model

Orders follow the six states named in the brief (Figure 2). The Worker's start event moves a confirmed order to `Processing` and its completion event to `Completed`. If the completion arrives first, both steps are applied together and the `Processing` step is recorded as implied. Every transition is stored in an ordered status history.

4. Work completed in Weeks 1–11

Table 2 lists what each block of weeks delivered and the test cases that prove it; the full list is in `docs/test-plan.md`.

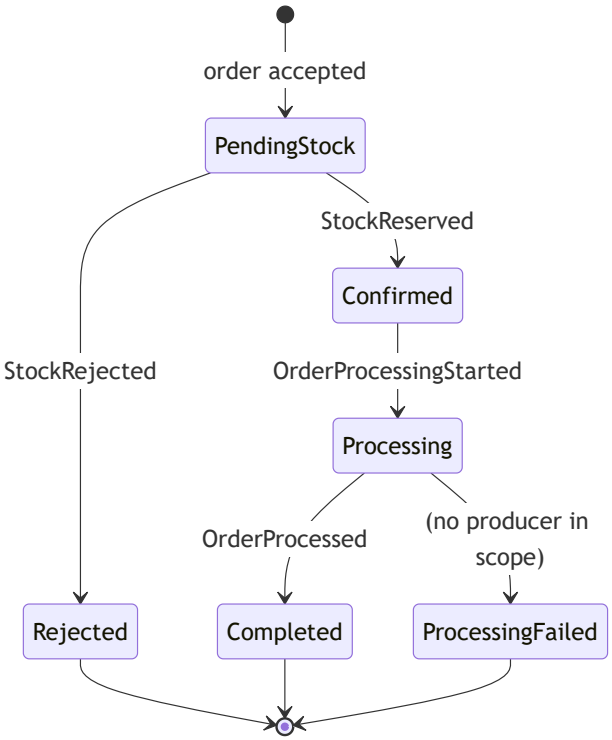


Figure 2: *Order states.* `ProcessingFailed` *is declared but has no producer, because the Worker is success-only in scope.*

5. Week 12: overload control and deployment

5.1 Load shedding (C3)

Four limits protect the system, all off by default so that C2 is unchanged (Figure 3). Nginx limits the rate of new orders and answers 429 [7]. The Order Service refuses a brand-new order before any database write, with 503 and `Retry-After`, when its Outbox backlog is full, when too many orders are still unfinished, or when its concurrency slots are taken. RabbitMQ main queues are bounded [5]. Every consumer takes at most 20 unacknowledged messages [4], so the bound covers real work, and a full queue only delays publication through the Outbox. A retry of an accepted order is answered before any application limit, but Nginx can still refuse it with 429, so clients retry with back-off [10].

5.2 Overload, drain and recovery

A three-phase JMeter run [8] offers 3 orders/s for 30 s, 60 orders/s for 60 s, then 3 orders/s for 60 s, with every limit on (Table 3). Counting all queued work showed that the first local run hid a backlog behind the Process Worker, which finishes about 3.7 orders/s. During the overload the Order Service accepted about 13 orders/s, orders waited

Table 2: *Work completed before Week 12.*

Weeks	Delivered	Evidence (test cases)
1–4	Scope, invariants and protocol; Compose stack; ERD generated from the code; C0 race reproduced; C1 never oversold below, at and above stock; API and event contracts; a named test plan.	TP-E01–E04, TP-B01–B04, TP-D01–D04
5–6	Order API with an Idempotency-Key (one order per key; a retry gets the original order); Transactional Outbox with mandatory publishing and publisher confirms.	TP-A01–A07, TP-O01, TP-O02
7–8	Inventory Service with a warm-up gate and Lua reservation under hot-product contention; a PostgreSQL failure after a Redis reservation is rebuilt without a second deduction; stock results drive `Confirmed` or `Rejected`; a correctness validator whose broken fixtures (oversold, unlogged deduction, reservation/order mismatch, double deduction, unsettled run, missing data, database unreachable) each fail or report inconclusive.	TP-I01–I03, TP-W01–W04
9–10	Inbox in every consumer that writes data (the stateless Worker relies on Order Service's); crash after commit before acknowledgement, and since the review a crash between the Redis reservation and the PostgreSQL save; concurrent and out-of-order duplicates; retry at 1, 2 and 4 s, then a dead-letter queue per service; reconciliation of stuck orders.	TP-M01, TP-M02, TP-F01, TP-F02
11	Deterministic Process Worker, the `Processing` state and its history; correlation IDs across the three services; metrics, a run-time collector and metric extraction.	TP-P01–P08

about 40 s end to end, and the backlog took 105 s to clear, failing the 60 s target. The new in-flight limit refuses what the Worker cannot finish: fewer orders are accepted (548 instead of 808), but each completes in about 6 s and the backlog clears in 6.9 s (4.5 s in the repeat run) (Figures 4 and 5). The same plan on EC2 with the assessed code accepted 582 orders, completed each in about 5.5 s and cleared the backlog in 6.2 s; every refused key was checked and none left an order.

In every run, the refusals matched JMeter, the Nginx log and the admission metric exactly; no refused request left an order (the local runs checked 10 of each kind, the 24 October EC2 run all 3,487); every accepted order completed with exactly one unit of stock; and no service restarted. With every limit switched off, 3,747 orders got in and the backlog did not drain in time, so the test detects missing limits.

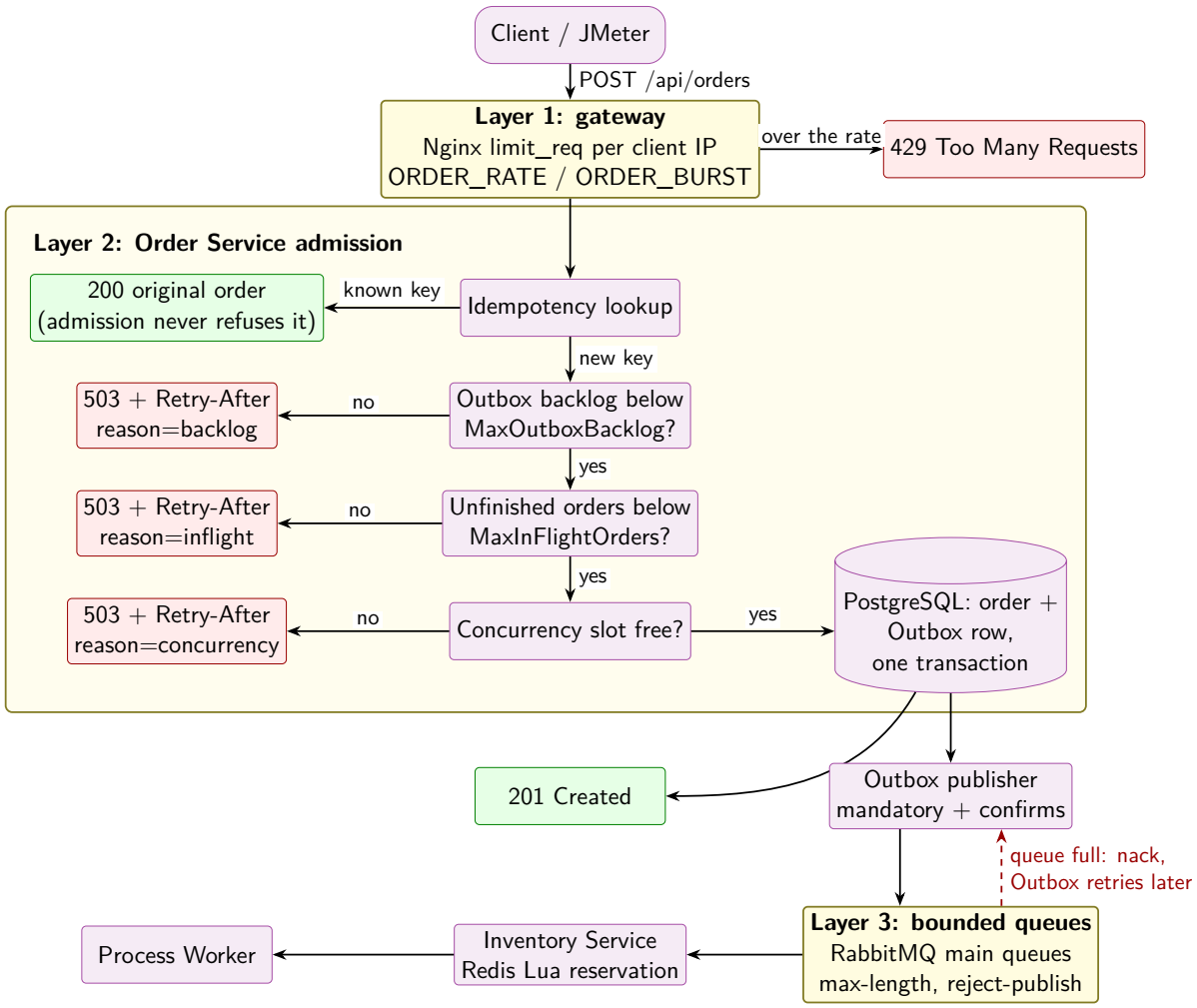


Figure 3: *Where each load-shedding layer refuses work.*

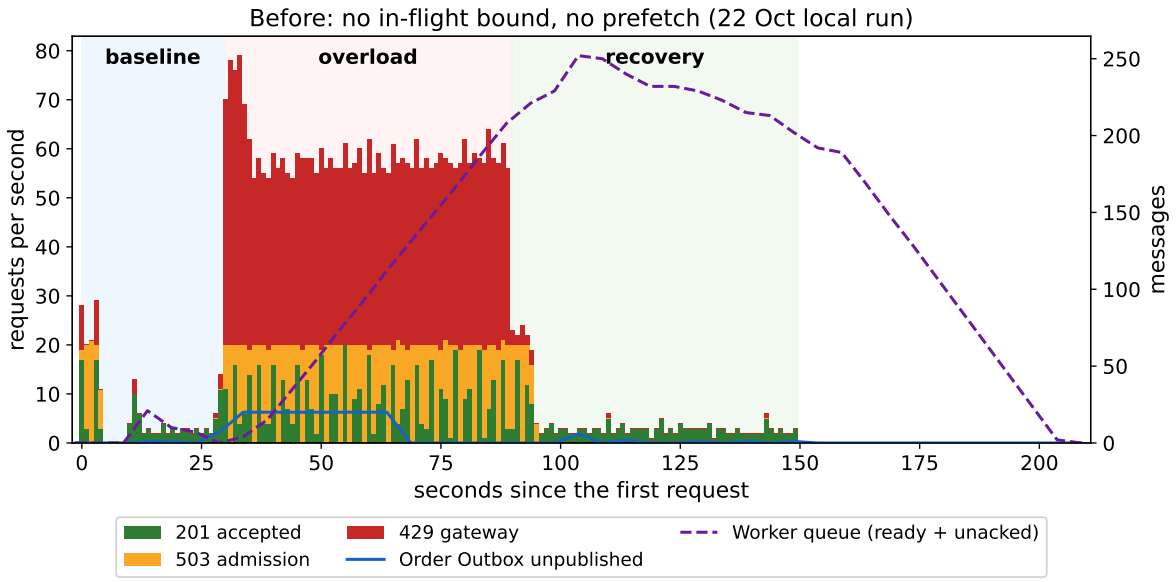


Figure 4: *Local overload run before the review fixes: the Worker queue grows to about 250 messages and drains only after the load stops.*

Table 3: *The same overload plan, one run each; official values come from Weeks 13–14.*

Overload run (pre-pilot limits)	Local 22 Oct (before)	Local 23 Oct (+ prefetch)	Local 23 Oct (+ in-flight)	EC2 24 Oct (assessed code)
Order submissions	3,995	4,027	4,084	4,069
Refused 429 / 503	2,390 / 832	2,403 / 816	2,464 / 1,072	2,445 / 1,042
Accepted, all completed	773	808	548	582
Accepted (201) response, p50 / p95	38 / 65 ms	–	27 / 39 ms	13 / 31 ms
Refused 429 / 503 response, p95	4 / 37 ms	–	3 / 25 ms	2 / 23 ms
Accepted to stock decision, p50 / p95	–	–	1.2 / 2.2 s	1.2 / 1.9 s
Accepted to Completed, p50 / p95	38 / 64 s	40 / 66 s	6.1 / 6.8 s	5.5 / 6.1 s
Backlog drain after overload	3.9 s[a]	105 s	6.9 s	6.2 s
Final drain after the last request	59 s	55 s	7.1 s	6.5 s
Recovery time[b]	7 s	5 s	7 s	6 s
Recovery-phase p95 latency	63 ms	55 ms	34 ms	19 ms

[a]The earlier, narrower measure, which missed messages already delivered to the Worker; not comparable with the other columns. [b]First 10 s window after the recovery phase starts with no refusals, no errors, p95 at most 1 s and at least one accepted order per second. Response times count POST submissions only, never status polls; "–" means the raw file needed is not archived. The EC2 value uses the second-pass definition (p95 of accepted responses only, no 1 s gap in the window). An earlier EC2 run on 22 October used the code from before the review and its raw data was not archived; it is superseded by this one.

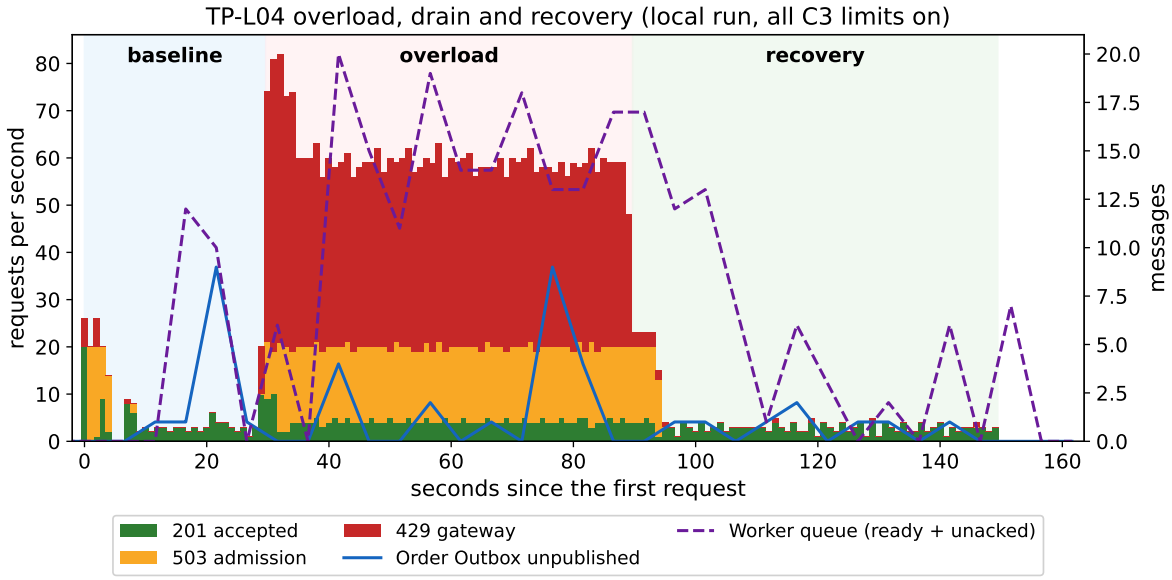


Figure 5: *Local overload run with the in-flight limit and prefetch: the Outbox and the Worker queue stay small and clear within seconds.*

5.3 AWS deployment

Terraform [11] creates two EC2 hosts in `ap-southeast-2`: a backend (`m7i-flex.large`) that runs the stack in Docker Compose with pinned images, and a separate JMeter host (`c7i-flex.large`) (Figure 6). Only Nginx is reachable, and only from the JMeter host. There is no SSH (AWS Systems Manager is used instead), no NAT, Elastic IP or load balancer, and both hosts terminate themselves after four hours. Three live sessions passed: only port 80 answered from the JMeter host, a JMeter smoke test ran 44 of 44 requests successfully, two overload runs completed (the second on the assessed code), and every session was destroyed with nothing left behind.

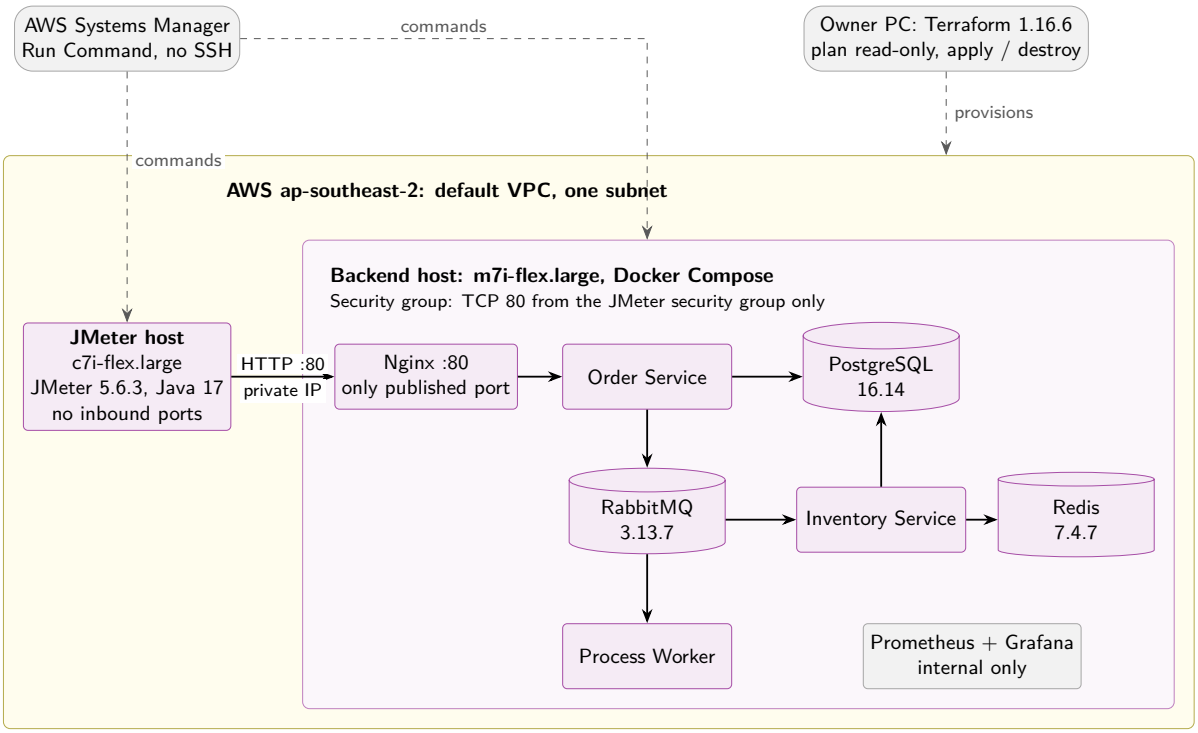


Figure 6: *The EC2 deployment.*

6. External review after Week 12

An external review graded Weeks 1–12 strictly. Every finding was confirmed in the code or the saved data before it was fixed, and each fix was re-tested:

- **State regression.** A late "processing started" event could move a `Completed` order back to `Processing`. The order state is now an optimistic-concurrency token; a late event re-reads and is acknowledged as stale. A unit test fails without the fix.
- **Hidden backlog.** No consumer limited its unacknowledged messages, so the Worker held its whole backlog outside the queue bound (peak 252 unacknowledged before, 20 after). The backlog measure now counts all queued work, and the in-flight limit

keeps it small; its test fails without the limit.

- **Measurement.** The metric extractor now counts a run's orders by their own keys (773, where a time window found 771), stops on any SQL error instead of reading it as 0, and counts recovery only when nothing is refused. The publication time of an event used to be stamped after its broker confirm, so a fast Inventory decision could precede it: 714 publish-to-decision intervals were negative. It is now the send time of the confirmed publish. Negative intervals are counted and reported, never set to zero; the run in Table 3 has none.
- **Reconciliation** now also recovers orders stuck in `Confirmed` or `Processing` (lost Worker work); its runtime test fails without the change.
- **Wording.** Statements that overclaimed were corrected, and order submissions are now reported apart from status polls.
- **Second pass.** The state race is also tested on PostgreSQL with two connections held at a barrier (both save orders, duplicate starts, conflicting stock results); four of the six cases fail without the concurrency token. A crash between the Redis reservation and the PostgreSQL save is now injected; the restarted consumer writes the missing result with no second deduction, and a control that erases Redis's record of the order deducts twice and fails. The Outbox publishers no longer sleep after a full batch. The EC2 Worker listened only on its own loopback, so its metrics were never scraped there; this is fixed, and the queue policy and the collector now work through `docker exec` without a published management port; all three were checked on EC2 on 24 October. The extractor judges recovery latency on accepted responses only and has fixtures for all-429, all-503, too little throughput, a gap in the window, missing samples and a failed database query. Outbox rows are claimed with a row lock, so Inventory replicas never publish the same row (400 rows give 400 messages with 2 or 4 replicas, 800 without the lock; TP-L05).

7. Verification

Every claim links to a saved log in `tests/evidence/`; `docs/evidence-index.md` maps each requirement of the brief to its test cases and logs. Three kinds of evidence are kept apart: offline checks that read code, documents and configuration (design alignment, requirement trace, deployment files); application tests that run the services on throwaway containers on the development PC; and live deployment evidence from the three EC2 sessions (port isolation, smoke test, two overload runs). All 22 verifiers pass in their latest runs (Figure 7): 21 were re-run on the assessed code on 24 October, and the runtime-accounts check (TP-E04) last ran on 23 October, before the second-pass

changes, none of which touches database accounts. The counts are read from the logs by the chart script, not typed in. The 104 unit tests also pass. Most verifiers were also run with a negative control that must make them fail. The chart counts checks; it does not rank how strong each verifier is.

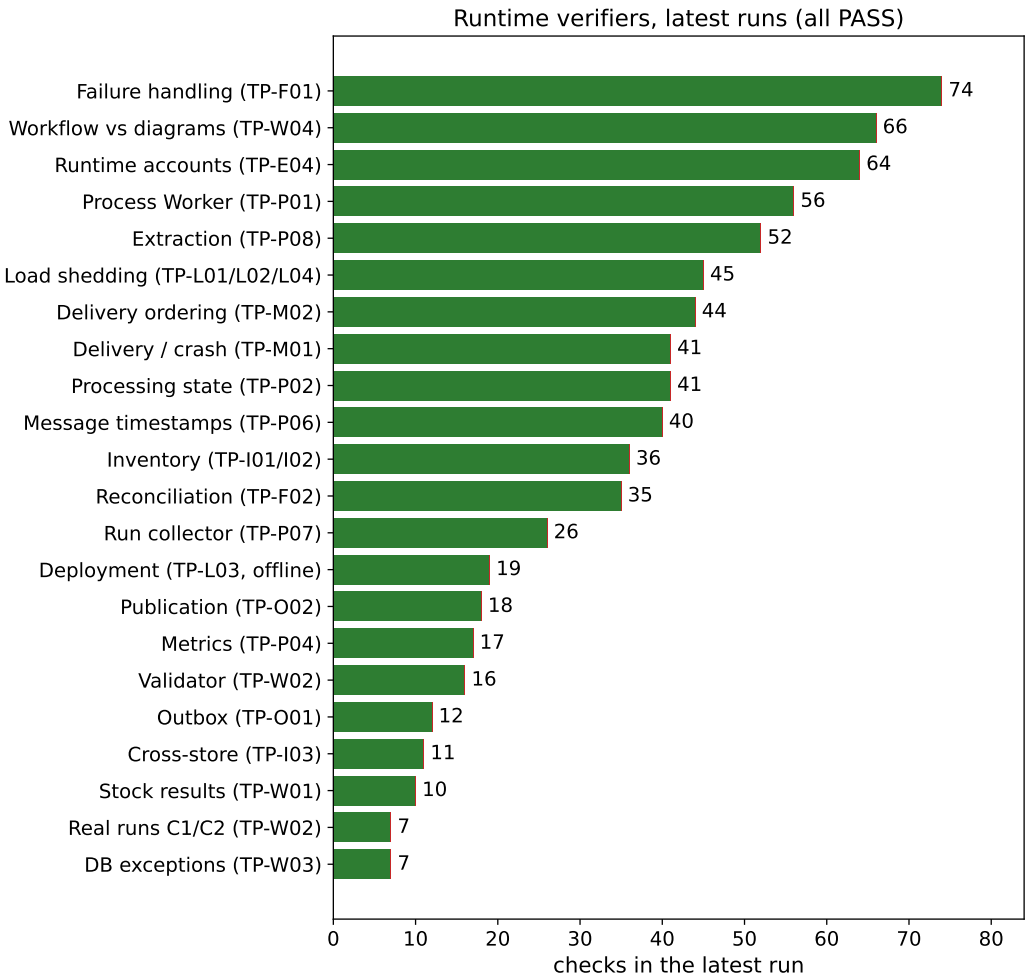


Figure 7: *Checks per verifier in its latest run.*

7.1 Assessed code and environment

All results in this report come from the code of the `main-test-bug-fix` branch at commit `eb04117`, built with the .NET SDK 10.0.300 (`global.json`). Local runs used Windows 11 with Windows PowerShell 5.1 and Docker Desktop (PostgreSQL 16.14, Redis 7.4.11, RabbitMQ 3.13.7 with the management plugin, Nginx 1.31.3; the verifiers start these from the `postgres:16`, `redis:7`, `rabbitmq:3-management` and `nginx:latest` tags, whose resolved versions are given here), JMeter 5.6.3 on Java 25, on one development PC. Unit tests ran with `dotnet test`; the six PostgreSQL race tests used a throwaway PostgreSQL 16 container. The EC2 sessions used `m7i-flex.large` (back-

end) and `c7i-flex.large` (JMeter) in `ap-southeast-2`, on the code from before the review, and a third on 24 October on the assessed code (Table 3); its raw JTL, collector samples and stage intervals are archived in `tests/evidence/`.

Table 4: *Requirements of the brief and the test cases that cover them (full map with log paths in `docs/evidence-index.md`).*

Requirement	Test cases
Accept, store and return an order ID; polling API	TP-A01–A07
Atomic reservation; stock never negative, never above initial stock, one reservation per order	TP-I01–I03, TP-B01–B02, TP-W02
Transactional Outbox, publisher confirms, routing	TP-O01, TP-O02
Inbox; duplicates and crashes have no second effect	TP-M01, TP-M02
Bounded retry, dead-letter queues, reconciliation	TP-F01, TP-F02
Process Worker; state history and four timestamps	TP-P01–P03
Metrics, collection and extraction	TP-P04, TP-P06–P08
Overload refused with control; backlog drains; no restart	TP-L01, TP-L02, TP-L04
Several Inventory replicas publish each event once (C4)	TP-L05
Terraform, EC2, separate JMeter host	TP-L03
Repeated experiments C0–C4	TP-X01, TP-X02 (not run; Weeks 13–14)

8. Research-question status

8.1 First local pilot: C1 against C2

To give the research question a first data point, C1 and C2 were offered the same steady load on the development PC, one after the other, on the same processes and database (`scripts/run-pilot-c1-c2.ps1`, `tests/jmeter/pilot-steady.jmx`). Each had its own product with 100,000 units, so every request could succeed; no load-shedding limit was on; the Worker ran 32 orders at a time (Table 6). The validator passed all four runs.

Table 5: *Configurations from the brief and their status.*

Config.	Purpose	Status	Evidence so far
C0	Naive read-then-write; shows the race only	Done	Over-selling reproduced (TP-B01)
C1	PostgreSQL atomic update; main baseline	Built, correct; local pilot	Never oversold (TP-B02); pilot at 50 and 200 r/s (Table 6)
C2	Redis Lua, RabbitMQ, Outbox and Inbox	Built, verified; local pilot	Correct under failures and duplicates; pilot against C1 (Table 6)
C3	C2 plus load shedding	Tested locally and on EC2	Overload, drain and recovery (Table 3)
C4	C3 with 1, 2 and 4 Inventory consumers	Not started	Week 13 pilot

Table 6: *Local pilot, one run per cell, 60 s each. Response times count POST submissions only. C1 answers with the stock decision; C2 answers with the acceptance and decides later.*

Offered load (target)	50 requests/s		200 requests/s	
	C1	C2	C1	C2
Achieved submissions/s	51.7	53.0	159.5[a]	202.0
Response p50 / p95 / p99 (ms)	17 / 84 / 1,723	25 / 43 / 84	1,129 / 1,778 / 2,149	31 / 61 / 121
Stock decisions/s[b]	52.7	52.1	161.2	70.8
Accepted to stock decision, p50 / p95	= response	1.8 / 2.8 s	= response	61.7 / 105.6 s
Accepted to Completed, p50 / p95	–	3.5 / 5.2 s	–	69.4 / 122.4 s
Errors / validator	0 / PASS	0 / PASS	0 / PASS	0 / PASS

[a]JMeter's 200 threads, each waiting about 1.1 s per C1 response, could not offer more; the offered rate for C1 was therefore below 200/s, so that pair is not an equal-load comparison. [b]C1: decision rows per second; C2: Inventory's decisions (its result rows) per second, the same event as "accepted to stock decision". Raw data: `tests/evidence/20261024T112852Z-pilot-c1-c2-50rps-*` and `tests/evidence/20261024T113202Z-pilot-c1-c2-200rps-*`.

At 50 requests/s both configurations keep up; C2 answers with lower tail latency, and its stock decision follows about 1.8 s later. At 200 requests/s they behave differently. C1's single conditional UPDATE on one row serialises the requests: it still decides about 160 orders/s, but each response waits about 1.1 s. C2 keeps answering in under 61 ms (p95), but its one Inventory consumer handles messages one at a time and decides only about 70 orders/s, so the decisions fall behind (published to decided: 60 s at the median).

The bottleneck of C2 on this PC is therefore the Inventory consumer, which is exactly what C4's extra consumers address. This is one run per cell on one machine shared with the load generator, so it shows the shape of the comparison, not official numbers.

8.2 Design limits that affect the comparison

The review found two design limits that would decide that comparison. Each Outbox publisher sent at most 50 events and then waited 2 s, so it stayed below 25 events/s, and the Process Worker finished about 3.7 orders/s. The Outbox limit is removed: a full batch is followed at once by the next and is sent on one channel with its confirms awaited together, and 300 queued events now drain in 1.3 s instead of at least 10 s. The Worker's limit is now a declared experiment parameter (`Processing:MaxConcurrency`, default 1; the pilot above used 32). As configured for the overload runs (Table 7), the Worker finishes about 4 orders/s and the in-flight bound admits only what it finishes, which is why those runs accepted about 4 orders/s. The pilot shows the next limit is the single Inventory consumer (about 70 decisions/s on this PC), far below the brief's target of about 2,000 reservation attempts per second, so the Week 13 pilot measures the stock path's capacity on EC2 and reports stock-decision latency apart from completion latency, as the brief requires. Adding Inventory replicas (C4) also adds Inventory Outbox publishers. They used to read unpublished rows without a row lock, so every replica published the same rows (800 messages for 400 rows with two replicas); C4 would have measured the duplication. Each publisher now claims its batch with `FOR UPDATE SKIP LOCKED`, and with 2 or 4 replicas each row is published exactly once (TP-L05). Each Inventory instance still handles one message at a time; letting one instance process several at once is a second lever, measured next to C4 in the pilot.

8.3 Pre-pilot settings

Every limit below is a pre-pilot value, chosen on the development PC; the Week 13 pilot sets the official values on EC2 and records them with each run.

9. Team contributions

The team states an equal split (50% each); it is the members' own statement, not derived from the repository history.

10. Limitations and threats to validity

- **One run per configuration.** Every number in Table 3 is a single run on one PC; no spread can be reported until the repeated runs of Week 14.
- **Development PC and one EC2 run.** The local runs share one machine between

Table 7: *Settings used in the local overload run (Table 3).*

Setting	Value
Nginx rate limit (POST `/api/orders`)	20 requests/s, burst 1 (default when off: 200/s, burst 100)
Admission: concurrent new orders	8
Admission: Order Outbox backlog	20 unpublished rows
Admission: unfinished orders	database baseline + 30
RabbitMQ main queues	max-length 5, overflow reject-publish
Consumer prefetch	20 per consumer
Process Worker	200 ms fixed delay, 1 order at a time
Outbox publisher	batches of 50, then a 2 s wait (the run predates the change in Section 6: a full batch is now followed at once)
Consumer retry	1, 2 and 4 s, then the service's dead-letter queue
Reconciliation	every 30 s; `PendingStock` after 30 s, `Confirmed`/`Processing` after 300 s; at most 3 re-sends
Load	3 orders/s for 30 s, 60/s for 60 s, 3/s for 60 s; one product, 10,000 units

Table 8: *Contributions of each member.*

Member	Student ID	Main contributions
Vũ Tấn Minh	523V0012	50% of the work, shared equally across all parts
Vũ Đình Kiệt	523V0011	50% of the work, shared equally across all parts

the load generator, the services and the broker. EC2 has one run on the assessed code (24 October, raw data archived); the earlier EC2 run of 22 October used the code from before the review and kept only its console summary (`tests/evidence/20261022T163505Z-ec2-overload.log`).

- **Fixed downstream delay.** The Worker's 200 ms delay is a stand-in for real work, so completion latency mostly measures this setting and the Worker's concurrency.
- **Redis persistence.** Reservation idempotency relies on Redis keeping its set of reserved orders. If Redis lost it, a reconciliation re-send could take a second unit for the same order (stock lost, never oversold). This failure is not injected.
- **Shared verifier database.** Verifiers reuse one throwaway PostgreSQL; rows from an interrupted run can disturb a later one. The reproduction guide starts from a fresh one.
- **Floating container tags in the verifiers.** The Compose files pin every image, but the verifiers' throwaway containers use tags such as `nginx:latest`; a later pull can change the versions above.

- **Windows-only verifiers and no continuous integration.** The verifiers are Windows PowerShell 5.1 scripts; the collector runs on Linux under PowerShell 7, but the verifiers have not.
- **Passing is not proof.** Each verifier checks the cases it names; most were also shown to fail on a deliberately broken variant, but none proves correctness for inputs it does not try.

11. Risks and mitigation

Table 9: *Main risks for Weeks 13–15.*

Risk	Effect	Mitigation
Throughput limits in the Worker and the single Inventory consumer (about 70 decisions/s in the local pilot)	C1 versus C2 decided by these limits, not by the architecture	Outbox drains continuously (done); Worker concurrency and the number of Inventory consumers set in the pilot; Outbox rows claimed with a lock (done, TP-L05); measure each stage's capacity first
Redis losing its record of reserved orders (no persistence guarantee is configured)	A re-sent order could take a second unit: stock lost, never oversold	Keep reconciliation re-sends rare; state the Redis persistence setting in the experiment record
EC2 time and credits; data lost when the stack is destroyed	Missing raw data for the report	Four-hour self-terminate timer; copy every run's raw data back before destroying (runbook updated)
Verifiers run only on Windows PowerShell 5.1; no continuous integration	Harder to reproduce on Linux and EC2	Port the verifiers to PowerShell 7 and add a build and unit-test pipeline
Pilot limits may not transfer from the PC to EC2	Over- or under-refusal in the official runs	Set every limit from the EC2 pilot and freeze it before the official runs

12. Next steps

- **Week 13 (31/10–06/11):** measure the stock path's capacity on EC2, with C4 replicas and with one Inventory instance processing several messages at once, at equal offered

loads; set the Worker's concurrency from the pilot; build the remaining workloads (ramp-up, steady, spike, hot product, skewed products, demand below, at and above stock); pilot C1–C4 with 1, 2 and 4 Inventory consumers on EC2; set every limit from the pilot; freeze source, configuration and the run matrix; write the Experimental Design chapter.

- **Week 14 (07/11–13/11):** run every configuration 3–5 times with reset, seed and warm-up; produce figures with percentiles, median and variation; complete Results, Discussion and Conclusion; reproduce a validated run from the README by 13 November.
- **Week 15 (14/11–21/11):** final checks, formatting and submission by 21 November.

References

[1] C. Richardson, *Microservices Patterns*. Shelter Island, NY: Manning, 2018.

[2] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

[3] RabbitMQ, "Consumer Acknowledgements and Publisher Confirms." `https://www.rabbitmq.com/docs/confirms`

[4] RabbitMQ, "Consumer Prefetch." `https://www.rabbitmq.com/docs/consumer-prefetch`

[5] RabbitMQ, "Queue Length Limit." `https://www.rabbitmq.com/docs/maxlength`

[6] Redis, "Scripting with Lua." `https://redis.io/docs/latest/develop/interactive-programmability/eval-intro/`

[7] NGINX, "Module ngx_http_limit_req_module." `https://nginx.org/en/docs/http/ngx_http_limit_req_module.html`

[8] Apache Software Foundation, "Apache JMeter User's Manual." `https://jmeter.apache.org/usermanual/index.html`

[9] Volosoft, "ABP Framework Documentation." `https://abp.io/docs/latest`

[10] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC: Pragmatic Bookshelf, 2018.

[11] HashiCorp, "AWS Provider documentation." `https://registry.terraform.io/providers/hashicorp/aws/latest/docs`